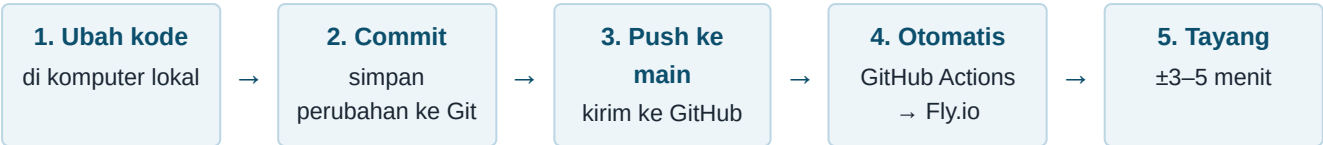


1. Ringkasan alur

Aplikasi ini **tidak perlu di-deploy manual**. Setiap kali kode di-*push* ke cabang `main` di GitHub, GitHub Actions otomatis membangun dan menerbitkannya ke Fly.io.



Komponen	Nilai
Repositori GitHub	jn1xia/perumda-ledger
Cabang produksi	main
Nama aplikasi Fly	perumda-ledger-scs9va
Wilayah server	Singapura (sin)
Alamat produksi	https://perumda-ledger-scs9va.fly.dev
Berkas alur otomatis	.github/workflows/fly-deploy.yml

2. Prasyarat (cukup sekali di awal)

Kebutuhan	Keterangan
Node.js 20.x	Wajib versi 20 — versi lain dapat menggagalkan <code>npm install</code> pada paket <code>sqlite3</code> .
Git	Untuk mengirim kode ke GitHub.
Akses tulis GitHub	Akun harus punya izin <i>push</i> ke repositori.
flyctl	Hanya diperlukan bila ingin deploy manual atau memeriksa status server (bab 5–7).

Menyiapkan token Fly di GitHub (sekali saja)

Alur otomatis memerlukan *secret* bernama `FLY_API_TOKEN` pada repositori. Bila belum ada, buat dengan langkah berikut:

```
# 1) buat token deploy dari komputer yang sudah login ke Fly
flyctl auth login
flyctl tokens create deploy --app perumda-ledger-scs9va
```

Salin token yang muncul, lalu di GitHub buka: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**, isi **Name** = `FLY_API_TOKEN` dan **Secret** = token tadi.

3. Cara 1 — Deploy otomatis lewat GitHub disarankan

Ini cara normal sehari-hari. Cukup kirim kode ke `main`:

```
# 1) pastikan berada di folder proyek dan di cabang main
cd perumda-ledger
git checkout main
git pull origin main          # ambil perubahan terbaru dulu

# 2) lihat berkas apa saja yang berubah
git status

# 3) tandai perubahan & simpan (commit)
git add .
git commit -m "perbaiki perhitungan laporan Juli"

# 4) kirim ke GitHub – ini yang memicu deploy
git push origin main
```

Setelah `git push` berhasil, deploy berjalan sendiri. Pantau prosesnya di GitHub: tab **Actions** → alur **Fly Deploy**. Bila tanda centang hijau muncul, perubahan sudah tayang.

Bila bekerja di cabang terpisah

```
# buat cabang kerja
git checkout -b perbaikan-neraca
... lakukan perubahan ...
git add .
git commit -m "perbaiki baris Neraca"
git push -u origin perbaikan-neraca
```

Push ke cabang selain `main` **tidak** memicu deploy. Perubahan baru tayang setelah cabang tersebut digabungkan (*merge*) ke `main` lewat Pull Request di GitHub.

4. Perubahan yang TIDAK memicu deploy

Alur otomatis sengaja mengabaikan perubahan yang hanya menyangkut dokumentasi, agar server tidak dibangun ulang tanpa perlu:

Pola berkas	Contoh	Memicu deploy?
docs/**	docs/REKAP_PENATAAN_STRUKTUR_COA.pdf	Tidak
**/*.md	README.md, CLAUDE.md	Tidak
Berkas lain	src/pages/LRA.jsx, server/routes/api.cjs	Ya

Jadi bila hanya menambah PDF ke folder `docs/`, jangan heran bila alur **Fly Deploy** tidak berjalan — itu memang perilaku yang diinginkan.

Menjalankan deploy secara manual dari GitHub

Bila perlu men-deploy ulang tanpa mengubah kode (misalnya setelah mengganti *secret*), buka **Actions** → **Fly Deploy** → **Run workflow** → **Run workflow**. Tombol ini tersedia karena alur memuat pemicu `workflow_dispatch`.

5. Cara 2 — Deploy manual dengan flyctl

Dipakai bila GitHub Actions sedang bermasalah, atau ingin menguji hasil build langsung dari komputer:

```
flyctl auth login
flyctl deploy --remote-only --app perumda-ledger-scs9va
```

`--remote-only` berarti proses build dikerjakan di server Fly, bukan di komputer lokal — tidak perlu memasang Docker.

Terminal terlihat “menggantung” selama beberapa menit itu normal. Terminal ditahan oleh proses build jarak jauh. Bila ragu, tunggu ± 3 menit lalu periksa nomor rilis dengan perintah pada bab 6.

6. Memastikan deploy berhasil

```
# daftar rilis – versi paling atas adalah yang aktif
flyctl releases --app perumda-ledger-scs9va

# kesehatan mesin (harus "started" dan "passing")
flyctl status --app perumda-ledger-scs9va

# log berjalan, untuk melihat pesan saat aplikasi menyala
flyctl logs --app perumda-ledger-scs9va
```

Selanjutnya buka `https://perumda-ledger-scs9va.fly.dev` dan tekan **Ctrl+Shift+R** (hard-refresh) supaya peramban memuat versi terbaru, bukan versi tersimpan di cache.

Catatan waktu nyala. Server diatur `auto_stop_machines` dan `min_machines_running = 0`, artinya mesin dimatikan otomatis saat tidak ada pengunjung dan menyala lagi saat diakses. Akses pertama setelah lama menganggur bisa terasa lambat beberapa detik — ini normal, bukan kerusakan.

7. Membatalkan perubahan (rollback)

Bila rilis terbaru bermasalah, kembali ke rilis sebelumnya:

```
# 1) lihat daftar rilis, catat kolom image rilis yang masih baik
flyctl releases --app perumda-ledger-scs9va

# 2) terbitkan ulang image lama tersebut
flyctl deploy --image <image-rilis-lama> --app perumda-ledger-scs9va
```

Alternatif melalui Git — membatalkan commit terakhir lalu push ulang (deploy otomatis berjalan lagi):

```
git revert HEAD
git push origin main
```

Gunakan `git revert`, bukan `git reset --hard`, agar riwayat perubahan tetap utuh dan tidak mengganggu rekan yang memakai repositori yang sama.

8. Peringatan penting soal basis data

Deploy dapat menimpa basis data produksi dalam kondisi tertentu. Saat aplikasi menyala, skrip `entrypoint.sh` memeriksa basis data pada volume dan akan **menyalin ulang basis data hasil build** apabila salah satu kondisi berikut terpenuhi:

- basis data produksi belum ada sama sekali;
- tabel `journal_lines` belum memiliki kolom `tanggal` (skema lama);
- tidak ditemukan satu pun jurnal multi-baris (3 baris atau lebih).

Basis data lama tetap dicadangkan ke berkas `.bak` sebelum ditimpa, namun **data yang diinput setelah build terakhir dapat hilang.**

Dalam keadaan normal ketiga kondisi di atas tidak terpenuhi, sehingga deploy **aman** dan data produksi tidak tersentuh. Meski begitu, **lakukan pencadangan sebelum deploy besar** lewat menu **Backup** pada aplikasi, atau:

```
curl -X POST https://perumda-ledger-scs9va.fly.dev/api/system/backup \
-H "X-User-Role: admin"
```

9. Yang dikerjakan saat proses build

Sebagai gambaran, inilah tahapan yang dijalankan Fly.io setiap kali deploy (sesuai `Dockerfile`):

No	Tahapan
1	Memasang dependensi (<code>npm install</code> , termasuk kompilasi <code>sqlite3</code>)
2	Membangun antarmuka React ke folder <code>dist/</code> (<code>npm run build</code>)
3	Membuat basis data awal dari skema + seed COA dan Aset Tetap
4	Mengimpor data laporan rujukan dari berkas Excel
5	Mengimpor ulang jurnal dengan dukungan multi-baris
6	Merapikan <code>journal_lines.akun_code</code> agar Buku Besar lengkap
7	Menjalankan <code>entrypoint.sh</code> lalu menyalakan server

10. Bila terjadi kendala

Gejala	Penanganan
Alur Fly Deploy tidak berjalan sama sekali	Periksa cabang — deploy hanya dari <code>main</code> . Bila perubahan hanya di <code>docs/</code> atau berkas <code>.md</code> , itu memang diabaikan (bab 4).
Actions gagal dengan pesan <code>Error: No access token available</code>	<code>Secret FLY_API_TOKEN</code> belum dibuat atau sudah kedaluwarsa — buat ulang (bab 2).
Build gagal pada <code>npm install / sqlite3</code>	Pastikan Node.js yang dipakai versi 20.x . Versi lain sering gagal mengompilasi <code>sqlite3</code> .
<code>git push</code> ditolak (<i>rejected</i>)	Ada perubahan baru di GitHub. Jalankan <code>git pull origin main</code> lebih dulu, selesaikan konflik bila ada, lalu push kembali.
Halaman masih menampilkan versi lama	Tekan Ctrl+Shift+R . Bila masih sama, pastikan nomor rilis sudah bertambah lewat <code>flyctl releases</code> .
Aplikasi tidak dapat diakses	Jalankan <code>flyctl status</code> dan <code>flyctl logs</code> untuk melihat penyebabnya; bila perlu kembalikan ke rilis sebelumnya (bab 7).

Ringkasan perintah harian: `git pull origin main` → `git add .` → `git commit -m "..."` → `git push origin main` → tunggu ±3–5 menit → hard-refresh peramban.

Panduan ini disusun mengikuti konfigurasi yang berlaku pada repositori per 31 Juli 2026 (`.github/workflows/fly-deploy.yml`, `fly.toml`, `Dockerfile`, `entrypoint.sh`). Bila konfigurasi berubah, panduan ini perlu disesuaikan.